

D01. Requirements Interpretation, Scope, and Assumptions

Project Context

LogistiCo is preparing a new distribution center and is implementing FlowCore, a simulated warehouse operations environment. FlowCore is not intended to be a full warehouse management system, but a compact technical simulation that allows a junior technical implementation team to demonstrate how a selected warehouse process can be configured, executed, validated, and documented in a realistic commissioning context.

Requirements Interpretation

Based on the project brief, the team understands the assignment as a small, coherent warehouse commissioning simulation rather than a full warehouse product. The solution has to run on team laptops, use free and publicly available tools, and combine the main bootcamp topics: Linux/Bash, configuration handling, Oracle SQL, PL/SQL, a small C++ component, testing, and documentation. For that reason, the team selected a narrow inbound scenario that can be implemented end-to-end, explained clearly, and checked through deterministic outputs. In this document, requirements interpretation means converting the general brief into one practical scope that the team can actually build and present.

Scope of the Project

The project scope is to develop a compact technical simulation for inbound warehouse commissioning. The simulation covers the selected process of receiving incoming goods, assigning them to valid warehouse locations, recording status or event changes, and performing validation checks to ensure the environment behaves correctly after configuration or logic changes. The project does not attempt to cover the whole warehouse lifecycle.

Problem Statement

The main problem addressed by the project is the current lack of a compact and controlled technical solution that can execute this inbound simulation. Such a solution is needed so that the team can validate configuration ideas, demonstrate technical readiness, and support early testing before production rollout. In other words, the problem is not a future warehouse exception such as overloading or congestion. The problem is the absence of a reliable simulation environment for early commissioning and validation work.

Assumptions

- Incoming goods are represented in a simplified way as inbound pallets or equivalent handling units.
- Warehouse locations are predefined.
- The simulation runs in a controlled technical environment.
- A small dataset is sufficient for demonstration purposes.
- Status transitions are simplified to match the selected inbound flow.
- Configuration and logic changes are limited to the selected project scope.

Project Boundaries

The project is limited to a simplified inbound warehouse commissioning simulation. The following areas are outside the selected scope:

- outbound order processing
- picking and dispatch operations
- full warehouse lifecycle coverage
- graphical user interface development
- real hardware integration
- external enterprise system integration
- advanced routing or optimization logic

Constraints

The project must satisfy the following constraints:

- all required software must be free to obtain and use
- the project must run on the team's laptops
- the solution must be small enough to complete within the timeframe of the training.
- the solution must remain aligned with the taught bootcamp technologies
- the project must not depend proprietary warehouse systems
- the project must not depend on instructor-provided infrastructure

Success Criteria

Success will be judged based on several connected criteria. First, the project must execute the selected inbound flow in a repeatable way, from receiving incoming goods to reaching the final stored state. Second, goods must be represented correctly as inbound pallets or equivalent handling units, assigned to valid warehouse locations according to the defined rules, and tracked through accurate status or event changes. Third, the solution must run on the team's laptops using only free and publicly available tools. It must also confirm its stability through validation checks that show the environment behaves as expected after configuration or logic changes, and it must produce clear technical evidence through generated outputs, logs, or database results. Finally, all required project artifacts must be complete and aligned with the defined deliverables.

